Overview

Many LLM products fail in production because teams optimize prompts but ignore inference systems engineering. At scale, user experience and cloud spend are driven by queueing, batching, KV-cache behavior, and hardware utilization.

This chapter covers practical inference engineering concepts needed for LLMOps: - prefill vs decode bottlenecks, - KV-cache lifecycle and memory pressure, - continuous batching and scheduling, - serving throughput/latency/cost trade-offs.

Inference Pipeline Fundamentals

Prefill vs decode

Generation typically has two phases: 1. **Prefill**: process input context and populate KV cache. 2. **Decode**: generate output token by token.

Prefill is usually compute-heavy for long prompts; decode is iterative and latency-sensitive.

Why latency feels worse than average metrics

Users experience tail latency. Even with strong average latency, long-tail requests can degrade UX and SLO compliance.

Operationally track: - P50 (typical), - P95/P99 (tail), - queue wait time, - token throughput.

KV Cache and Memory Economics

KV cache stores attention keys/values for generated context.

Implications: - memory grows with sequence length and concurrency, - inefficient allocation reduces effective batch capacity, - cache strategy directly affects throughput.

Cache strategies

- static reservation (simple but wasteful),
- dynamic/paged allocation (higher utilization),
- prefix caching for repeated context reuse.

Batching and Scheduling

Static batching

Collect requests into fixed batches. Simple but can underutilize hardware when requests have uneven lengths.

Continuous (inflight) batching

Admits new requests as slots free during decode. This usually improves utilization and throughput in variable-length traffic.

Scheduling policies

- FIFO fairness,
- shortest-job-first style heuristics,
- priority classes (premium traffic, low-latency paths).

Every policy has fairness vs efficiency trade-offs.

Serving Architecture Patterns

Pattern 1: Single model API for moderate traffic

- one serving stack,
- fixed autoscaling policy,
- simple observability.

Pattern 2: Router + model pool

- request classifier chooses model tier,
- cheap model for easy tasks,
- expensive model for hard tasks.

Pattern 3: Hybrid serving

- local or self-hosted models for predictable tasks,
- external API fallback for spikes or special capabilities.

Cost and Quality Trade-Offs

Major cost levers: - prompt length, - output token caps, - model size/tier routing, - caching hit rate, - batching efficiency.

Quality levers: - better retrieval/context selection, - constrained decoding/output schema, - eval-informed prompt templates.

A common mistake is increasing model size to fix what is actually context or orchestration debt.

Capacity Planning and SLO Design

Define SLOs explicitly: - response latency target (P95), - availability target, - quality floor metrics, - cost budget per request.

Then model traffic with: - arrival rates, - burst factor, - prompt length distribution, - output length distribution.

Capacity planning should be revisited after each major prompt or routing change.

Observability for Inference Systems

Minimum telemetry: - request count and concurrency, - queue depth/wait, - prefill/decode time split, - tokens in/out, - cache hit/miss and cache pressure, - error and timeout rates.

Useful derived metrics: - cost per successful response, - throughput per GPU, - tail-latency by segment (feature, customer tier, prompt class).

Production Case Studies & Exceptions

Case 1: Support assistant with long-context prompts

Issue: good answer quality but poor tail latency. Fix pattern: context compression + prefix caching + output caps. Exception: aggressive truncation can harm factual grounding.

Case 2: High-volume content generation

Issue: cost explosion after growth. Fix pattern: tiered routing (small model default, large model fallback), strict budget guardrails. Exception: misclassification in router can hurt quality if fallback triggers too late.

Case 3: Enterprise analytics copilot

Issue: periodic queue spikes at reporting windows. Fix pattern: burst autoscaling + priority scheduling for interactive users. Exception: batch backfills may starve if priority policy is not balanced.

Interview Questions & Answers

1. **Q:** Prefill vs decode?
   A: Prefill processes input context and builds KV cache; decode generates tokens iteratively.
2. **Q:** Why does KV cache matter?
   A: It is a major memory bottleneck that determines concurrency and throughput.
3. **Q:** Static vs continuous batching?
   A: Static uses fixed batches; continuous injects requests dynamically for better utilization.
4. **Q:** Why monitor P95 not only average latency?
   A: Tail latency drives user pain and SLO breaches.
5. **Q:** What is prefix caching?
   A: Reusing cached attention states for repeated prompt prefixes.
6. **Q:** First three telemetry signals you add?
   A: Queue wait, tokens in/out, and request latency percentiles.
7. **Q:** How to reduce cost quickly?
   A: Limit token budgets, improve routing, and cut redundant context.
8. **Q:** Common false fix for latency?
   A: Jumping to larger hardware before fixing prompt/context inefficiency.
9. **Q:** Why separate prefill and decode timings?
   A: They have different bottlenecks and optimization strategies.
10. **Q:** What is a serving router?
    A: A component selecting model tier/path per request based on policy.
11. **Q:** When does small model routing fail?
    A: When request complexity detection is weak and hard cases are misrouted.
12. **Q:** What metric links performance and finance?
    A: Cost per successful response at target quality.
13. **Q:** Why can longer prompts hurt concurrency?
    A: They increase prefill work and cache footprint.
14. **Q:** Why include timeout budgets per stage?
    A: To isolate queueing, inference, and downstream integration failures.
15. **Q:** What is a practical rollback in serving?
    A: Revert model version or routing policy flag to last stable config.
16. **Q:** How do you prevent runaway generation costs?
    A: Hard max token limits and adaptive stop criteria.
17. **Q:** Why do priority classes matter?
    A: They protect critical interactive traffic under load.
18. **Q:** What is the risk of over-prioritization?
    A: Starving lower-priority queues and creating hidden SLA debt.
19. **Q:** One must-have capacity planning input?
    A: Real traffic distribution of prompt and completion lengths.
20. **Q:** One-line production advice?
    A: Treat LLM serving as queueing + memory engineering, not only model engineering.

References

- vLLM docs: https://docs.vllm.ai/
- PagedAttention paper: https://arxiv.org/abs/2309.06180
- TensorRT-LLM key features: https://nvidia.github.io/TensorRT-LLM/key-features.html
- NVIDIA vLLM overview: https://docs.nvidia.com/deeplearning/frameworks/vllm-release-notes/overview.html